

Tkinter for Beginners

A growing reference guide built alongside the Number Guessing Game project

1. What is tkinter?

tkinter is Python's built-in library for making desktop windows with buttons, text boxes, labels, and other visual elements. It comes included with most Python installations, so there is nothing extra to download. It isn't the fanciest-looking option, but it's simple, reliable, and perfect for learning how graphical programs work.

In tkinter, every visual element (a button, a label, a text box) is called a **widget**. Building a UI mostly means creating widgets, placing them in a window, and telling them what to do when the user interacts with them.

2. The big idea: event-driven programming

A terminal program and a window program are in control in different ways:

Terminal program	Window (GUI) program
Your code controls the order of events.	The user controls the order of events.
It asks a question with <code>input ()</code> and waits for the answer.	It shows a window and waits for any click or key press.
A <code>while True:</code> loop keeps the game running.	<code>window.mainloop ()</code> keeps the window running.
Output is shown with <code>print ()</code> .	Output is shown by updating widgets on screen.

In a GUI, your job is to connect actions to code: *"when this button is clicked, run this function."* Each click, key press, or other user action is called an **event**, and the function that runs in response is often called an **event handler** or **callback**. This style of programming is called **event-driven programming**.

3. Checking that tkinter is installed

Run this in your terminal (use `python3` if that's what you normally type):

```
python -m tkinter
```

If a small test window pops up, tkinter is ready. If you get an error, the most common fixes are:

System	Fix
Windows	Re-run the Python installer, choose <i>Modify</i> , and make sure <i>tcl/tk</i> and <i>IDLE</i> is ticked.

System	Fix
macOS	Install Python from python.org, which includes tkinter. With Homebrew: <code>brew install python-tk</code>
Linux (Ubuntu/Debian)	<code>sudo apt install python3-tk</code>

4. Your first window

Every tkinter program starts with the same four building blocks.

Importing tkinter

```
import tkinter as tk
```

This imports the library and gives it the short nickname `tk`, so you can write `tk.Button` instead of `tkinter.Button`. Almost all tkinter code you find online uses this nickname.

Creating the main window

```
window = tk.Tk()
```

`tk.Tk()` creates the main window (sometimes called the **root** window) and stores it in a variable. You'll use this variable whenever you want to change the window or put widgets inside it. Note the spelling: capital **T**, lowercase **k**.

Setting the title and size

```
window.title("My App")
window.geometry("400x300")
```

`title()` sets the text shown in the window's title bar. `geometry()` sets the window's size as "widthxheight" in pixels, written as a single string with a lowercase `x` between the numbers and no spaces. Both are optional; without them, tkinter picks a default title and size.

Starting the main loop

```
window.mainloop()
```

This opens the window and keeps it running, constantly listening for events like clicks and key presses. It works like a built-in `while True:` loop that tkinter manages for you. The loop ends when the user closes the window.

Watch out: `mainloop()` must be the **last line** of your program. Any code written after it will not run until the window is closed.

Putting it together

```
import tkinter as tk

window = tk.Tk()
window.title("My App")
window.geometry("400x300")

window.mainloop()
```

Running this opens an empty 400 by 300 pixel window titled *My App*. The pattern is always the same: import, create the window, set it up (and later, add widgets), then start the main loop.

5. Widgets: the building blocks of a UI

A **widget** is any visual element in a window. You create one by calling its name from `tk`, the same way you created the window with `tk.Tk()`. This project uses three widgets:

Widget	What it is	Example
<code>tk.Label</code>	Displays text the user can read but not edit.	Instructions, hints, the result message
<code>tk.Entry</code>	A single-line box the user can type into.	Where the player types a guess
<code>tk.Button</code>	A clickable button.	"Guess" and "New Game"

Creating a widget

```
greeting = tk.Label(window, text="Hello there!")
```

The **first argument** is always the widget's **parent**: the thing it lives inside. For now that's always `window`. After that come **options**, written as `name=value`, that control how the widget looks and behaves. You store the widget in a variable so you can change it or read from it later.

Useful options

Option	Works on	What it does
<code>text="..."</code>	Label, Button	The text shown on the widget.
<code>font=("Arial", 14)</code>	Label, Button, Entry	Font name and size, written as a tuple in brackets.
<code>width=10</code>	Label, Button, Entry	Width measured in characters, not pixels.
<code>justify="center"</code>	Entry, Label	Aligns text "left", "center", or "right".
<code>fg="blue" / bg="white"</code>	Label, Button, Entry	Text colour (foreground) and background colour.

A Label, an Entry, and a Button

```
title_label = tk.Label(window, text="What is your name?", font=("Arial", 12))
name_box = tk.Entry(window, font=("Arial", 14), width=15)
ok_button = tk.Button(window, text="OK")
```

A Button made this way appears on screen but does nothing when clicked yet. Connecting buttons to code is covered in a later step.

Multi-line text in a Label

A Label can show several lines of text. There are two ways to break text onto new lines.

1. Use `\n` inside the string wherever you want a new line:

```
tk.Label(window, text="Line one\nLine two\nLine three")
```

2. Use `wraplength` to let tkinter wrap long text automatically once it reaches a certain width (in pixels):

```
tk.Label(window, text="A long sentence that keeps going...", wraplength=280)
```

Multi-line text is centred by default. Add `justify="left"` to line it up on the left, which is easier to read for instructions or rules:

```
rules = (
    "How to play:\n"
    "1. I'm thinking of a number from 1 to 100.\n"
    "2. You have 5 guesses to find it."
)
tk.Label(window, text=rules, justify="left", font=("Arial", 11)).pack(pady=10)
```

Tip: Python automatically joins strings that sit next to each other inside brackets, so the rules example above is one single string. Splitting it across lines just makes it easier to read and edit in your code.

Bold and italic text

Add a third item to the `font` tuple to change the style:

```
tk.Label(window, text="Guess That Number", font=("Arial", 16, "bold"))
tk.Label(window, text="Good luck!", font=("Arial", 11, "italic"))
```

Available styles include `"bold"`, `"italic"`, and `"underline"`, and you can combine them, e.g. `"bold italic"`.

Watch out: More text needs more room. If your window cuts off widgets at the bottom, make it taller in `window.geometry()`.

6. Placing widgets with pack()

Creating a widget does **not** make it appear. You also have to place it in the window. The simplest way is `pack()`, which stacks widgets from top to bottom in the order you pack them, centred horizontally.

```
title_label.pack()  
name_box.pack()  
ok_button.pack()
```

Tip: If a widget isn't showing up, the first thing to check is whether you called `.pack()` on it.

Adding space around widgets

```
title_label.pack(pady=10)  
ok_button.pack(pady=5, padx=20)
```

`pady` adds empty space above and below a widget (in pixels), and `padx` adds space to its left and right. Padding is what stops a window from looking cramped.

Create and pack in one line

If you never need to touch a widget again after creating it (like a fixed heading), you can create and pack it in a single line without a variable:

```
tk.Label(window, text="Welcome!").pack(pady=10)
```

Watch out: Never write `name_box = tk.Entry(window).pack().pack()` `pack()` returns `None`, so `name_box` would store `None` instead of the widget, and using it later causes an error. If you need the widget later, create it on one line and pack it on the next.

Complete example

```
import tkinter as tk  
  
window = tk.Tk()  
window.title("Name Asker")  
window.geometry("300x180")  
  
tk.Label(window, text="What is your name?", font=("Arial", 12)).pack(pady=10)  
  
name_box = tk.Entry(window, font=("Arial", 14), width=15)  
name_box.pack()  
  
ok_button = tk.Button(window, text="OK")  
ok_button.pack(pady=10)  
  
window.mainloop()
```

7. Making a button do something

In a window program, nothing happens until the user does something. To make a button react to a click, you write a **function** containing the code you want to run, then connect it to the button with the command option. A function used this way is called a **callback** or **event handler**: tkinter "calls it back" whenever the event (the click) happens.

```
def say_hello():
    print("Hello! The button was clicked.")

hello_button = tk.Button(window, text="Say hello", command=say_hello)
hello_button.pack()
```

Every time the button is clicked, tkinter runs `say_hello()`. Here, the message is printed in the terminal, not in the window, which is a handy way to test that a button is connected.

No brackets after the function name

Notice it's `command=say_hello`, **not** `command=say_hello()`. The difference matters:

You write	What it means
<code>command=say_hello</code>	"Here is the function. Run it later, whenever the button is clicked." This is what you want.
<code>command=say_hello()</code>	"Run the function <i>right now</i> , while building the window, and give the button whatever it returns." The function runs once at startup, and clicking does nothing.

Tip: Think of `say_hello` as handing someone a recipe, and `say_hello()` as cooking the meal yourself right now. The button needs the recipe.

Define the function first

Python reads your file from top to bottom. If the button line comes before the `def` line, Python doesn't know the function exists yet and raises a `NameError`. A good layout for a tkinter program is:

```
import tkinter as tk

# 1. Functions
def say_hello():
    ...

# 2. Window setup
window = tk.Tk()

# 3. Widgets
...

# 4. Main loop
window.mainloop()
```

8. Changing a widget after it's created

In a terminal program you show new information with `print()`. In a window, you change what an existing widget displays using `.config()`:

```
result_label.config(text="Too high!")
```

This replaces the label's text on screen instantly. `config()` accepts the same options you used when creating the widget, so you can also change colours, fonts, and more:

```
result_label.config(text="You win!", fg="green")
result_label.config(font=("Arial", 14, "bold"))
```

Putting it together, a button can update a label when clicked:

```
import tkinter as tk

def show_message():
    message_label.config(text="The button was clicked!")

window = tk.Tk()
window.geometry("300x150")

message_label = tk.Label(window, text="Waiting...")
message_label.pack(pady=20)

tk.Button(window, text="Click me", command=show_message).pack()

window.mainloop()
```

Tip: This is why widgets you want to change later must be stored in a variable. The function needs a name to refer to, like `message_label`.

9. Reading what the user typed

To find out what's in an Entry box, call its `.get()` method. Like `input()`, it always returns a **string**, even if the user typed digits, so convert it with `int()` before comparing it with numbers.

```
text = guess_box.get()      # e.g. "42" (a string)
number = int(text)          # 42 (an int)
```

Watch out: Call `.get()` **inside** the function that runs when the button is clicked. If you call it at the top level of your program, it runs once when the window first opens, before the user has typed anything, so you'll always get an empty string.

Clearing the Entry box

```
guess_box.delete(0, tk.END)
```

This deletes everything from the first character (position 0) to the end (`tk.END`). It's a nice touch after each guess so the player doesn't have to erase their previous number.

Stopping a function early with return

In a loop you used `continue` to skip ahead after bad input. Inside a button's function there's no loop, because tkinter calls the function once per click. Instead, use `return`, which exits the function immediately. The next click simply runs it again from the top.

```
def check_guess():
    try:
        guess = int(guess_box.get())
    except ValueError:
        result_label.config(text="Please enter a whole number.")
        return      # stop here; nothing below runs

    result_label.config(text=f"You guessed {guess}")
```

10. Changing variables inside a function: global

Variables created outside any function (at the top level of your file) are called **global** variables. A function can **read** a global variable freely. But if the function tries to **change** one, Python assumes you meant to create a brand-new local variable that only exists inside the function, and you get an error:

```
count = 0

def add_one():
    count += 1      # UnboundLocalError!
```

To tell Python "I mean the global one," add a `global` line at the start of the function:

```
count = 0

def add_one():
    global count
    count += 1    # works: changes the global count
```

Watch out: Assigning with a plain `=` is sneakier than `+=`. Without `global`, a line like `count = 0` inside a function raises **no error at all**. Python quietly creates a new local variable that disappears when the function ends, and the global one never changes. If a function seems to "reset" something but nothing actually resets, a missing `global` is the first thing to check.

Inside a function, you want to...	Need global?
Read a global variable (e.g. compare with <code>secret_number</code>)	No
Call a method on a widget (e.g. <code>result_label.config(...)</code>)	No
Change a global variable (e.g. <code>attempts += 1</code> or <code>secret_number = ...</code>)	Yes

Tip: In a game, variables that must survive between clicks (the secret number, the attempt count) are kept as globals, because a function's local variables are wiped out every time it finishes. Larger programs often use classes instead, but globals are perfectly fine for a small project like this.

Why can a function use a widget created below it?

A function's code doesn't run when Python reads the `def`; it only runs when the function is called. By the time the user clicks the button, every widget has been created, so the function can use them. The "define before use" rule applies to **running** code: the button needs the function to exist when the button is created, but the function only needs the widgets to exist when it's called.

11. Disabling and enabling widgets

Sometimes a widget shouldn't be usable, for example the Guess button after the game has ended. Every Button and Entry has a state option you can change with `.config()`:

```
guess_button.config(state="disabled")    # greyed out, clicks do nothing
guess_button.config(state="normal")     # usable again
```

Widget	When disabled
Button	Appears greyed out. Clicking it does nothing, so its command never runs.
Entry	Appears greyed out. The user can't type in it, and your code can't change its text either.

Watch out: A disabled Entry ignores `.delete()` and `.insert()` without showing an error. If you need to clear it, either clear it **before** disabling it, or set `state="normal"` first, clear it, then disable it again if needed.

You can also set the state when creating a widget, e.g. `tk.Button(window, text="Next", state="disabled")` for a button that should only become usable later.

Colours to show success or failure

Changing a label's text colour with `fg` makes results easier to notice at a glance:

```
result_label.config(text="You win!", fg="green")
result_label.config(text="Game over!", fg="red")
result_label.config(text="Too high!", fg="black")
```

Colours stay until you change them again, so if you turn a label red, remember to set it back (e.g. to "black") when showing normal messages. Tkinter understands many colour names, such as "orange" and "dark green", as well as hex codes like "#2B6CB0".

Multi-line results with f-strings

`\n` works inside f-strings too, which is handy for showing two pieces of information in one label:

```
result_label.config(text=f"Too low!\nGuesses left: {guesses_left}")
```

12. Resetting: a New Game button

To let the player start again without closing the window, add a second button whose function puts everything back the way it was at the start. A reset function usually has three jobs:

Job	Example
Reset the game's variables (needs <code>global</code> , since they're being changed)	<code>secret_number = random.randint(1, 100)</code>
Re-enable any disabled widgets	<code>guess_button.config(state="normal")</code>
Reset what's shown on screen	<code>result_label.config(text="Good luck!", fg="black")</code>

```
def new_game():
    global secret_number, attempts
    secret_number = random.randint(1, 100)
    attempts = 0

    guess_box.config(state="normal")    # enable BEFORE clearing
    guess_box.delete(0, tk.END)
    guess_button.config(state="normal")
    result_label.config(text="Good luck!", fg="black")

new_game_button = tk.Button(window, text="New Game", command=new_game)
new_game_button.pack()
```

You can list several variables on one global line, separated by commas.

Tip: A good way to check a reset function: compare it with the top of your program. Every variable you set up at the start, and every widget whose text, colour, or state changes during the game, should be put back in the reset function.

Putting the cursor in the Entry box: focus()

```
guess_box.focus()
```

This places the typing cursor in the Entry box, so the player can type straight away without clicking it first. Call it once after creating the Entry (so it's ready when the window opens), and again at the end of your reset function.

Avoiding repeated code with helper functions

If the same few lines appear in several places (like disabling the button and entry whenever the game ends), you can move them into their own small function and call it wherever it's needed:

```
def end_game():
    guess_button.config(state="disabled")
    guess_box.config(state="disabled")

# later, wherever the game ends:
end_game()
```

If you ever need to change what happens at the end of a game, you then only change it in one place.

Bonus: pressing Enter to submit

Besides button clicks, tkinter can react to key presses using `.bind()`. This makes the Enter key run the same function as the Guess button:

```
guess_box.bind("<Return>", lambda event: check_guess())
```

"<Return>" is tkinter's name for the Enter key. When a key is pressed, tkinter passes an **event** object (with details like which key was pressed) to the function. `check_guess` doesn't accept any

arguments, so the `lambda event: check_guess()` part is a tiny one-line function that accepts the event, ignores it, and calls `check_guess()`.

Tip: Pressing Enter while the Entry is disabled does nothing, because a disabled Entry doesn't receive key presses, so the game can't continue after it ends.

13. Common beginner mistakes

Mistake	What happens	Fix
Forgetting <code>window.mainloop()</code>	The window flashes and disappears, or never appears.	Add it as the last line.
Writing <code>tk.tk()</code> or <code>tk.TK()</code>	<code>AttributeError</code>	The correct spelling is <code>tk.Tk()</code> .
Writing <code>"400 x 300"</code> or <code>"400X300"</code>	An error about a bad geometry specifier.	No spaces, lowercase x: <code>"400x300"</code>
Naming your file <code>tkinter.py</code>	Python imports your file instead of the real library, causing confusing errors.	Rename your file (and delete any <code>__pycache__</code> folder next to it).
Putting code after <code>mainloop()</code>	That code only runs after the window closes.	Set everything up before calling it.
Forgetting <code>.pack()</code>	The widget exists but never appears.	Call <code>.pack()</code> on every widget.
<code>box = tk.Entry(window).pack()</code>	<code>box</code> is <code>None</code> , causing errors later.	Create the widget on one line, pack it on the next.
Forgetting <code>window</code> as the first argument	The widget may end up in the wrong place, or an extra window appears.	Always pass the parent first: <code>tk.Label(window, ...)</code>
<code>command=my_function()</code>	The function runs once at startup; clicking does nothing.	Remove the brackets: <code>command=my_function</code>
Defining the function below the button	<code>NameError</code> : the name isn't defined.	Put all functions near the top, before the widgets.
Calling <code>.get()</code> outside the function	Always returns an empty string.	Call it inside the button's function.
Comparing <code>.get()</code> directly with a number	The string is never equal to the number, or a <code>TypeError</code> .	Convert first: <code>int(box.get())</code>
Changing a global without <code>global</code>	<code>UnboundLocalError</code>	Add <code>global</code> name at the top of the function.
Assigning a global with <code>=</code> without <code>global</code>	No error, but the global never changes; a hidden local copy is changed instead.	Add <code>global</code> name at the top of the function.
Using <code>continue</code> in a button function	<code>SyntaxError</code> : 'continue' not properly in loop.	Use <code>return</code> to exit the function early.

Mistake	What happens	Fix
Clearing a disabled Entry	Nothing happens; the old text stays.	Clear it before disabling, or set <code>state="normal"</code> first.
Changing a label's colour and never changing it back	Later messages stay red (or green).	Set <code>fg</code> back to your normal colour with each normal message.
Forgetting to reset something in the New Game function	Old values carry over, e.g. attempts keep counting from the last game.	Reset every variable and widget that changes during a game.

14. Quick reference

Code	What it does
<code>import tkinter as tk</code>	Imports tkinter with the nickname tk.
<code>window = tk.Tk()</code>	Creates the main window.
<code>window.title("text")</code>	Sets the title bar text.
<code>window.geometry("WxH")</code>	Sets the window size in pixels.
<code>tk.Label(window, text="...")</code>	Creates a text label.
<code>tk.Entry(window)</code>	Creates a text box the user can type in.
<code>tk.Button(window, text="...")</code>	Creates a button.
<code>widget.pack(pady=10)</code>	Places a widget in the window, with spacing.
<code>\n</code> in text	Starts a new line inside a Label.
<code>wraplength=280</code>	Wraps a Label's text automatically at 280 pixels.
<code>justify="left"</code>	Left-aligns multi-line text.
<code>font=("Arial", 16, "bold")</code>	Bold (or italic/underline) text.
<code>tk.Button(..., command=func)</code>	Runs func each time the button is clicked.
<code>label.config(text="...")</code>	Changes a widget's text (or other options) after creation.
<code>box.get()</code>	Returns the Entry's text as a string.
<code>box.delete(0, tk.END)</code>	Clears an Entry box.
<code>return</code>	Exits a function early.
<code>global name</code>	Lets a function change a global variable.
<code>w.config(state="disabled")</code>	Greys out a Button or Entry so it can't be used.
<code>w.config(state="normal")</code>	Makes a widget usable again.
<code>label.config(fg="red")</code>	Changes a label's text colour.
<code>global a, b, c</code>	Declares several globals on one line.
<code>box.focus()</code>	Puts the typing cursor in an Entry box.
<code>window.mainloop()</code>	Opens the window and keeps it running. Always last.

Where to go from here

You've built a complete tkinter program: widgets, layout, button callbacks, reading input, updating the screen, disabling widgets, and resetting state. Some ideas for practising further: track the player's best score across games, add difficulty levels (1-10, 1-100, 1-1000) with extra buttons, show a list of previous guesses, or try a new project from scratch, like a tkinter version of rock, paper, scissors or a simple calculator.